LAB 7

Fuzzing for Input Validation Bugs

Software Engineering Lab

Name: CHAITHANYA.N
SRN: PES1UG24CS124

GitHub repository: github.com/chaithanya2810/SE-Labs-PES1UG24CS124

1. Objective

The objective of this lab was to explore fuzz testing as a security and quality assurance technique, use the Hypothesis library to identify input-validation bugs in the supplied Python application, and improve the application so that unexpected or invalid inputs are handled safely.

2. Application under test

The application, credit_card_rewards_parser.py, contains three functions:

- mask_card_number() – masks a card number while retaining the last four digits.
- parse_transaction() – parses a transaction line containing a date, category and amount.
- parse_statement_total() – extracts the numeric statement total from a text line.

3. Test approach

The application was first executed with valid sample inputs. The test file, test_credit_card_rewards_parser.py, was then run using pytest and Hypothesis. Hypothesis generated text and None inputs for the three functions and reduced failing inputs to small examples that made the errors reproducible. After identifying the failures, the application was updated with input-type checks, validation and safe handling of invalid values. The tests were run again against the fixed application.

4. Fuzz testing results

The initial fuzz tests exposed crashes caused by unexpected inputs:

- mask_card_number(): None caused an AttributeError.
- parse_transaction(): an empty string caused an IndexError, and None caused an AttributeError.
- parse_statement_total(): ':' caused a ValueError; an empty string caused an IndexError; and None caused an AttributeError.

The functions were hardened to validate their inputs and return None for invalid input instead of raising an exception. Additional tests checked expected valid outputs and invalid-input handling.

4.1 Original application execution

```
PS D:\SE-Lab-Work\SE-Labs-PES1UG24CS124\Lab7> python credit_card_rewards_parser.py
==== Running mask_card_number ====
Enter a card number (e.g. 4111 1111 1111 1234): 4111 1111 1111 1234
Masked card number: ************1234

==== Running parse_transaction ====
Enter a transaction as DATE,CATEGORY,AMOUNT (e.g. 2026-11-05,dining,1550): 2026-11-11,dining
,1550
Category and amount: ('dining', 1550)

==== Running parse_statement_total ====
Enter the statement total line (e.g. Total: 12,500): Total: 12,500
Statement total: 12500
PS D:\SE-Lab-Work\SE-Labs-PES1UG24CS124\Lab7>
```

Figure 1. Output of the original application with valid sample inputs.

4.2 Fuzz test failure – card number

```
PS D:\SE-Lab-Work\SE-Labs-PES1UG24CS124\Lab7> python -m pytest -q --tb=short test_credit_card_rewards_parser.py::test_mask_card_number_no_crash
F
=================================== FAILURES ===================================
_____________________ test_mask_card_number_no_crash _____________________
test_credit_card_rewards_parser.py:13: in test_mask_card_number_no_crash
    mask_card_number(s)
credit_card_rewards_parser.py:6: in mask_card_number
    digits = number.replace(' ', '').replace('-', '')
             ^^^^^^^^^^^^^^
E   AttributeError: 'NoneType' object has no attribute 'replace'

During handling of the above exception, another exception occurred:
test_credit_card_rewards_parser.py:11: in test_mask_card_number_no_crash
    def test_mask_card_number_no_crash(s):
                   ^^^
test_credit_card_rewards_parser.py:15: in test_mask_card_number_no_crash
    assert False, f"mask_card_number crashed with input {s!r}: {e}"
E   AssertionError: mask_card_number crashed with input None: 'NoneType' object has no attribute 'replace'
E   assert False
E   Falsing test case: test_mask_card_number_no_crash(
E       s=None,
E   )
=========================== short test summary info ============================
FAILED test_credit_card_rewards_parser.py::test_mask_card_number_no_crash - AssertionError: mask_card_number crashed with input None: 'NoneType' object has no attribute 'replace'
1 failed in 2.38s
PS D:\SE-Lab-Work\SE-Labs-PES1UG24CS124\Lab7>
```

Figure 2. Hypothesis identified an AttributeError for a None input.

4.3 Fuzz test failure – transaction

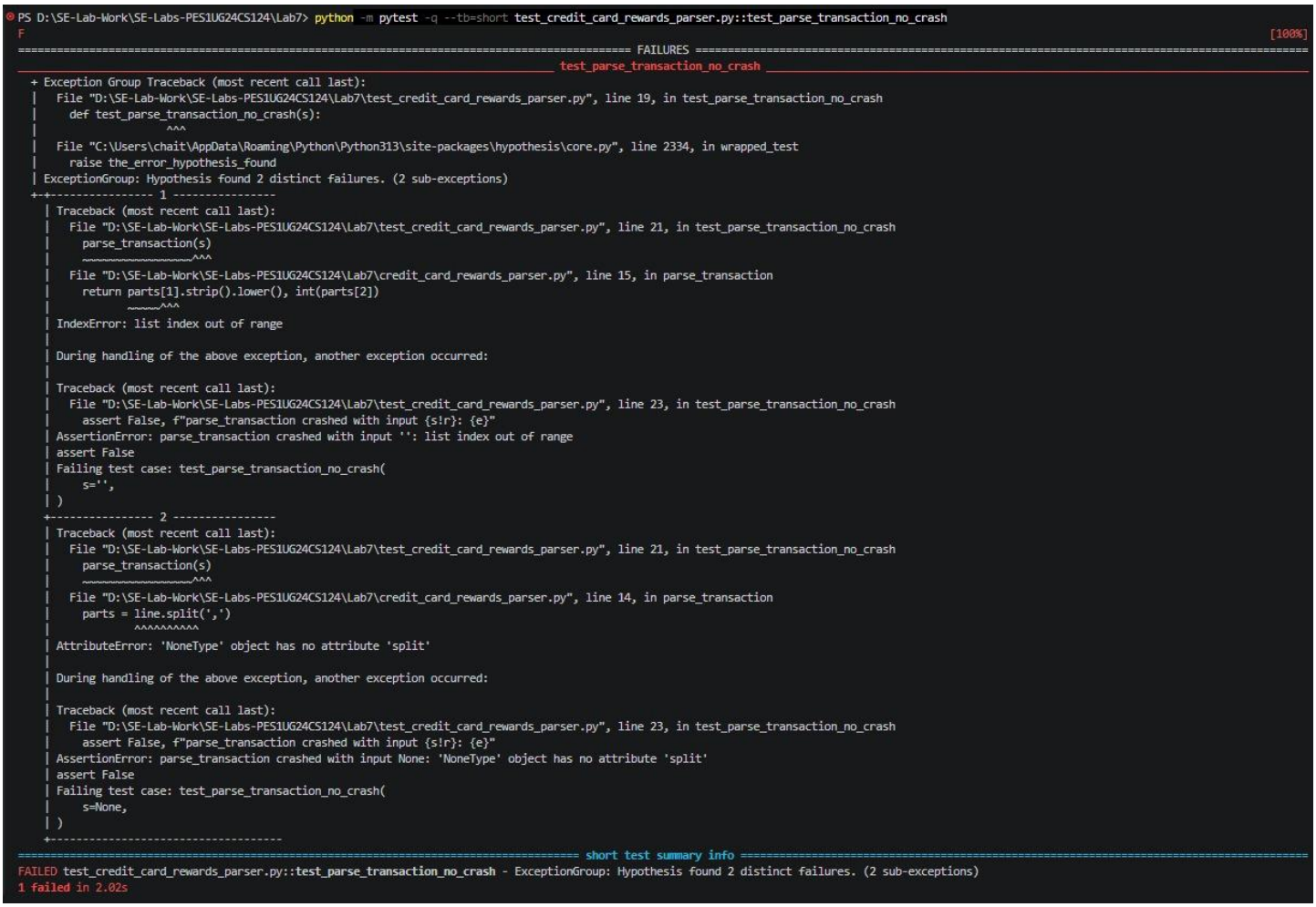


Figure 3. Hypothesis identified failures for empty-string and None inputs.

4.4 Fuzz test failure – statement total



Figure 4. Hypothesis identified failures for malformed, empty and None inputs.

4.5 Fixed application execution

```
PS D:\SE-Lab-Work\SE-Labs-PES1UG24CS124\Lab7> python credit_card_rewards_parser.py
==== Running mask_card_number ====
Enter a card number (e.g. 4111 1111 1111 1234): 4111 1111 1111 1234
Masked card number: ************1234

==== Running parse_transaction ====
Enter a transaction as DATE,CATEGORY,AMOUNT (e.g. 2026-11-05,dining,1550): 2026-11-11,dining,1550
Category and amount: ('dining', 1550)

==== Running parse_statement_total ====
Enter the statement total line (e.g. Total: 12,500): Total: 12,500
Statement total: 12500
PS D:\SE-Lab-Work\SE-Labs-PES1UG24CS124\Lab7>
```

Figure 5. Output of the fixed application using valid sample inputs.

4.6 Final test execution

```
PS D:\SE-Lab-Work\SE-Labs-PES1UG24CS124\Lab7> python -m pytest -q test_credit_card_rewards_parser.py
......
6 passed in 2.78s
PS D:\SE-Lab-Work\SE-Labs-PES1UG24CS124\Lab7>
```

Figure 6. Final pytest result: all 6 tests passed.

5. Result and conclusion

Fuzz testing with Hypothesis exposed input-validation bugs in all three functions. After defensive input validation was added, the application executed successfully with the sample inputs and the updated test suite completed successfully. The final test run reported 6 passed. This lab demonstrated how property-based fuzz testing can discover edge cases that may not be covered by manually selected test inputs.

6. Files submitted

- credit_card_rewards_parser.py – fixed application script.
- test_credit_card_rewards_parser.py – updated Hypothesis and additional test cases.
- PDF report containing the screenshots of the original run, failing fuzz tests, fixed run and passing tests.